

Gestión de Estado Local, Seguridad y Control de Acceso

Persistencia de sesión en el frontend

La persistencia de sesión se gestiona mediante el uso del objeto `localStorage`, el cual funciona como un almacén de tipo clave-valor dentro del navegador del cliente.

Durante el proceso de autenticación ocurre lo siguiente:

- El usuario envía sus credenciales al backend.
- El backend valida dicha información.
- Si la validación es correcta, el backend emite un JSON Web Token (JWT).
- El token incluye:
 - Identidad del usuario.
 - Rol asignado.
 - Marca temporal de expiración.
- El frontend almacena el token junto con el rol del usuario en `localStorage`.

Gracias a este mecanismo, el estado de autenticación se mantiene incluso si el usuario recarga el navegador, ya que el token persiste en el almacenamiento local.

Desde el punto de vista técnico, esta estrategia permite mantener la sesión sin utilizar cookies tradicionales. En su lugar, el token se envía en cada solicitud HTTP mediante la cabecera:

`Authorization: Bearer <token>`

Este enfoque reduce los riesgos asociados a ataques CSRF cuando se configura correctamente el envío de cabeceras.

Control de acceso preliminar en la interfaz

El rol almacenado en el frontend se emplea para realizar un control de acceso preliminar desde la interfaz gráfica. Esto permite:

- Renderizar u ocultar botones según los privilegios del usuario.
- Evitar interacciones inválidas desde la interfaz.

Este control es únicamente preventivo y visual. La validación definitiva de permisos ocurre siempre en el servidor.

Autenticación Stateless con JWT

El sistema implementa autenticación sin estado (*stateless authentication*) mediante JSON Web Tokens. En este modelo:

- El servidor no mantiene una tabla de sesiones activas.
- Cada solicitud entrante debe presentar un token válido.

Cada JWT encapsula:

- La identidad del usuario.
- El rol asignado.
- Un tiempo de expiración.

Cuando el cliente realiza una petición al backend:

- Envía el token en la cabecera **Authorization**.
- El backend verifica:
 - La firma criptográfica del token.
 - La vigencia temporal.
 - La integridad del contenido.

Si el token es válido, la solicitud se procesa. En caso contrario, se rechaza por motivos de autenticación.

Escalabilidad del sistema

Debido a que el servidor no almacena sesiones, múltiples instancias del backend pueden validar tokens sin necesidad de sincronizar estados internos. Esto mejora la escalabilidad horizontal del sistema y permite distribuir la carga de trabajo entre varios servidores.

Expiración del token

La expiración del token está configurada en 60 minutos. Este mecanismo:

- Limita el tiempo de uso de un token en caso de interceptación.
- Reduce la superficie de ataque.
- Obliga a la reautenticación periódica del usuario.

Decisión Tecnológica: Base de Datos

Se seleccionó PostgreSQL como sistema gestor de base de datos debido a sus capacidades avanzadas en:

- Control de concurrencia mediante MVCC (Multi-Version Concurrency Control).
- Integridad referencial mediante claves foráneas.
- Soporte completo de transacciones ACID.

A diferencia de motores embebidos como SQLite, PostgreSQL permite soportar múltiples usuarios concurrentes sin riesgo de corrupción de datos, lo cual resulta esencial en un entorno institucional con múltiples operadores simultáneos.

Seguridad y Control de Acceso (Backend)

Encriptación: Protección de Credenciales

En el backend se implementa un mecanismo de protección de contraseñas mediante la librería `passlib` utilizando el algoritmo `bcrypt`.

Bcrypt introduce dos propiedades fundamentales:

- **Salting automático:** inserta valores aleatorios en cada hash, evitando ataques por tablas arcoíris.
- **Cost factor:** incrementa el tiempo de cómputo por intento, mitigando ataques de fuerza bruta.

Desde una perspectiva criptográfica, las contraseñas no se almacenan como texto plano sino como funciones hash unidireccionales, lo que implica que no existe una operación inversa para recuperar la contraseña original.

Esto garantiza que, incluso ante una intrusión a la base de datos, la información sensible permanezca protegida a nivel matemático.

Control de Acceso Basado en Roles (RBAC)

El modelo de autorización se implementa mediante Role-Based Access Control (RBAC).

Cada usuario posee un rol lógico definido en el sistema, como:

- Director.
- Coordinador.
- Usuario estándar.

En cada endpoint protegido, el backend inspecciona el rol contenido en el JWT antes de ejecutar la operación solicitada. Este proceso se realiza mediante middleware de validación.

Si el rol no cumple con los privilegios requeridos:

- La operación se cancela.

Conclusión técnica

El sistema integra autenticación sin estado mediante JWT, persistencia de sesión en el frontend, cifrado seguro de credenciales y control de acceso basado en roles. Estas decisiones permiten garantizar seguridad, escalabilidad y control adecuado de privilegios dentro del sistema.